

实验一 Quartus 软件与状态机第一部分

一、实验目的

- (1) 掌握软件流程, 掌握状态机编程;
- (2) 掌握时序仿真的基本方法。
- (3) 掌握时序电路的基本分析方法;
- (4) 掌握从模型到工程实际的电路变化;

二、实验原理与步骤

1. 分析 sync 代码功能。

Out1 输出尝试解决输入信号怎么进入同步时序电路, Out3 尝试解决输入数据的获取时刻和计数方法, 而 Out2 时长不定, 还可能因为延迟而丢失脉冲信号, 因此不能使用。sync 电路即是“同步 synchronize”, 用来将输入进机器的信号与内部信号进行同步, 从而能正常使用外部输入信号。

2. 以 tlight2 工程代码为例, 学习 VHDL 语言的逻辑语言描述, D 触发器的设计和计数器的设计。

带异步复位的 D 触发器代码补充:

```
process(rst, clk)

begin

    if rst = '1' then          -- 异步复位 (假设高电平有效)

        q <= '0';             -- 复位时输出置 0
```

```

        elsif clk'event and clk = '1' then    -- 时钟上升沿触发

            q <= d;                            -- 正常工作时输出数据

        end if;

    end process;

```

模 N 计数器代码：

```

if rst = '0' then

    q <= (others => '0');

elsif rising_edge(clk) then

    if q = N-1 then

        q <= (others => '0');

    else

        q <= q +1;

    end if;

end if;

```

3. 学习仿真交通灯工程，以 tlight28_sim.bdf 为顶层文件，进行仿真分析，查找错误。

先分析交通灯问题，对状态机文件 states.vhd 进行学习。因为不能同时有两条道路通行，我们将所有情况分解为四种状态：东西通行 s0、东西转南北通行 s1（两个黄灯）、南北通行 s2、南北转东西通行 s3（两个黄灯）。

状态 s0 (东西绿灯): 东西方向通行，南北方向禁止：

1. 如果南北方向有车且东西方向也有车，等待 60 秒后转换到状态 s1
2. 如果南北方向有车但东西方向无车，立即转换到状态 s1

状态 s1 (东西黄灯): 东西方向黄灯，南北方向红灯:

1. 等待 3 秒后转换到状态 s2

状态 s2 (南北绿灯): 南北方向通行，东西方向禁止:

1. 如果南北方向有车且东西方向也有车，等待 40 秒后转换到状态 s3
2. 如果南北方向无车，立即转换到状态 s3

状态 s3 (南北黄灯): 南北方向黄灯，东西方向红灯:

1. 等待 3 秒后转换到状态 s0

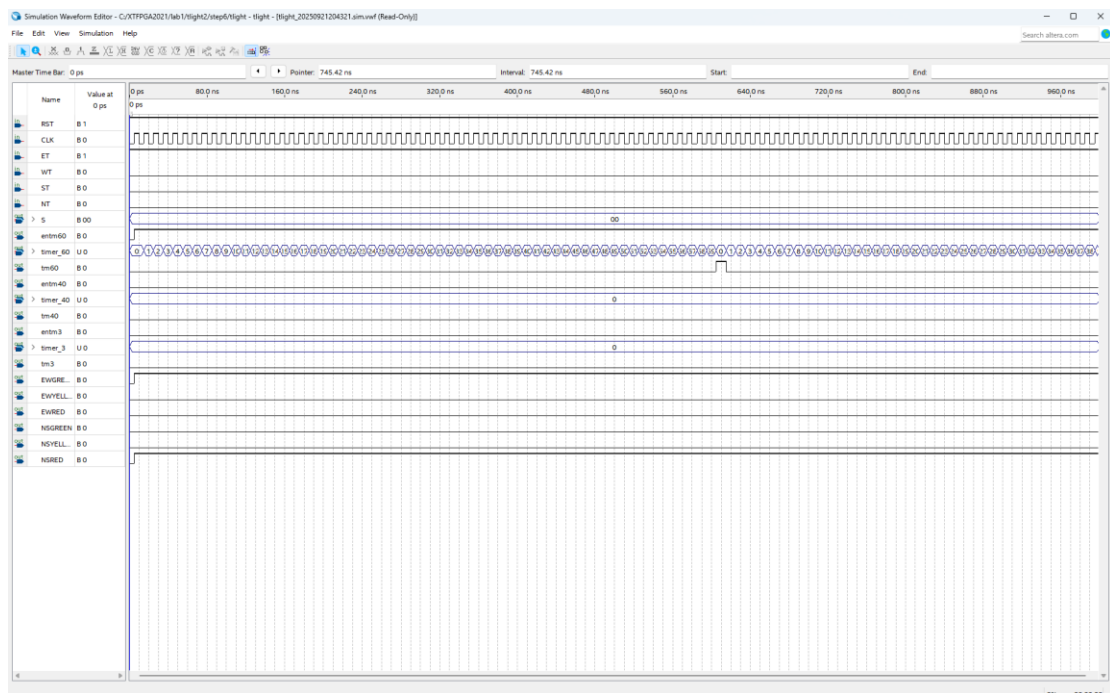
代码分析如下:

```

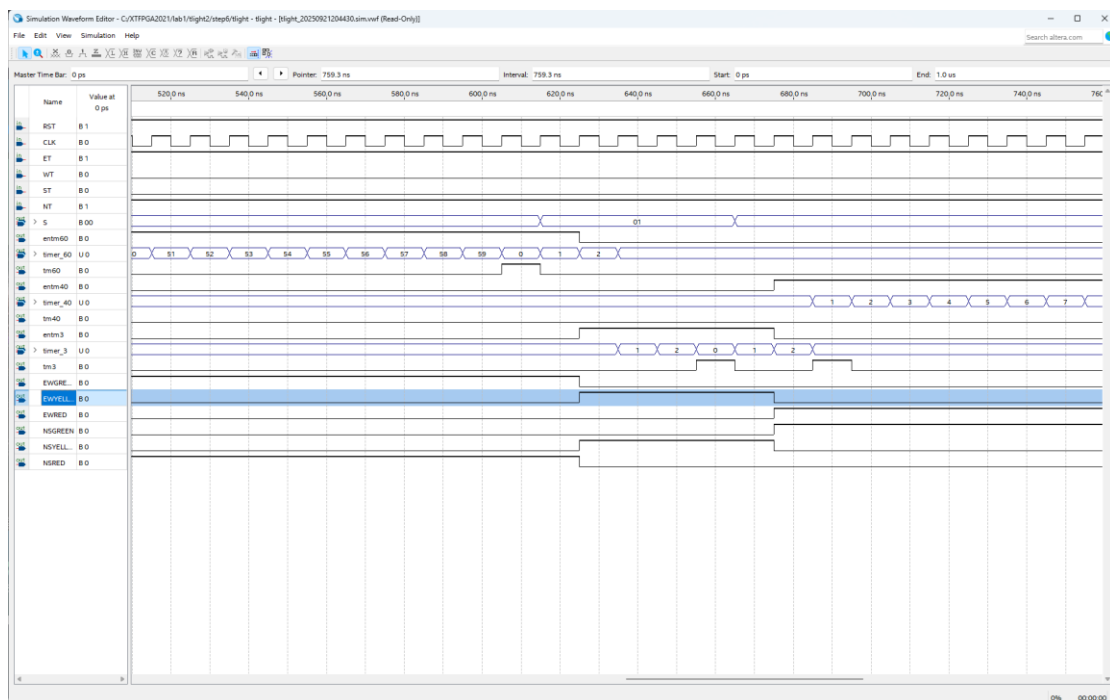
1  library IEEE;
2  use IEEE.std_logic_1164.all;
3  use ieee.std_logic_arith.all;
4  use ieee.std_logic_unsigned.all;
5  entity states is
6  port (
7      rst      : in std_logic;      -- 复位信号，低电平有效
8      clk      : in std_logic;      -- 时钟信号
9      nscar    : in std_logic;      -- 南北方向有车检测信号（1表示有车）
10     ewcar    : in std_logic;      -- 东西方向有车检测信号（1表示有车）
11     tm60     : in std_logic;      -- 60秒定时完成信号（1表示时间到）
12     tm40     : in std_logic;      -- 40秒定时完成信号（1表示时间到）
13     tm3      : in std_logic;      -- 3秒定时完成信号（1表示时间到）
14     s        : out std_logic_vector(1 downto 0) -- 状态机输出信号
15 );
16 end states;
17
18 -- 架构定义：状态机的行为描述
19 architecture states_behav of states is
20     constant s0 : std_logic_vector(1 downto 0) := "00"; -- 状态0：东西方向绿灯（主道通行）
21     constant s1 : std_logic_vector(1 downto 0) := "01"; -- 状态1：东西方向黄灯（转换状态）
22     constant s2 : std_logic_vector(1 downto 0) := "11"; -- 状态2：南北方向绿灯（副道通行）
23     constant s3 : std_logic_vector(1 downto 0) := "10"; -- 状态3：南北方向黄灯（转换状态）
24     signal ss : std_logic_vector(1 downto 0); -- 内部状态寄存器
25
26 begin
27     -- 状态转换进程
28     process(rst, clk)
29     begin
30         if rst = '0' then -- 异步复位，低电平有效
31             ss <= s0; -- 复位后进入状态0(东西绿灯)
32         elsif clk'event and clk = '1' then -- 时钟上升沿触发
33             case ss is
34                 when s0 => -- 当前状态：东西绿灯
35                     if nscar = '1' then -- 如果南北方向有车
36                         if ewcar = '1' then -- 如果东西方向也有车
37                             if tm60 = '1' then -- 60秒定时到
38                                 ss <= s1; -- 转换到状态1(东西黄灯)
39                             end if;
40                         else -- 东西方向无车，但南北方向有车
41                             ss <= s1; -- 立即转换到状态1(东西黄灯)
42                         end if;
43                     else -- 如果南北方向无车，保持状态0(东西绿灯)
44                     end if;
45                 when s1 => -- 当前状态：东西黄灯(转换状态)
46                     if tm3 = '1' then -- 3秒定时到
47                         ss <= s2; -- 转换到状态2(南北绿灯)
48                     end if;
49                 when s2 => -- 当前状态：南北绿灯
50                     if nscar = '1' then -- 如果南北方向有车
51                         if ewcar = '1' then -- 如果东西方向也有车
52                             if tm40 = '1' then -- 40秒定时到
53                                 ss <= s3; -- 转换到状态3(南北黄灯)
54                             end if;
55                         else -- 南北方向无车
56                             ss <= s3; -- 立即转换到状态3(南北黄灯)
57                         end if;
58                     else -- 南北方向无车
59                         ss <= s3; -- 立即转换到状态3(南北黄灯)
60                     end if;
61                 when s3 => -- 当前状态：南北黄灯(转换状态)
62                     if tm3 = '1' then -- 3秒定时到
63                         ss <= s0; -- 转换到状态0(东西绿灯)
64                     end if;
65                 when others => -- 其他未定义状态
66                     ss <= s0; -- 安全机制，回到状态0
67             end case;
68         end if;
69     end process;
70
71     s <= ss; -- 将内部状态输出到端口
72 end states_behav;

```

在仿真时先仿真最简单情况，即只有一条路上有车，此时该向车道绿灯恒亮，另一条车道红灯恒亮。



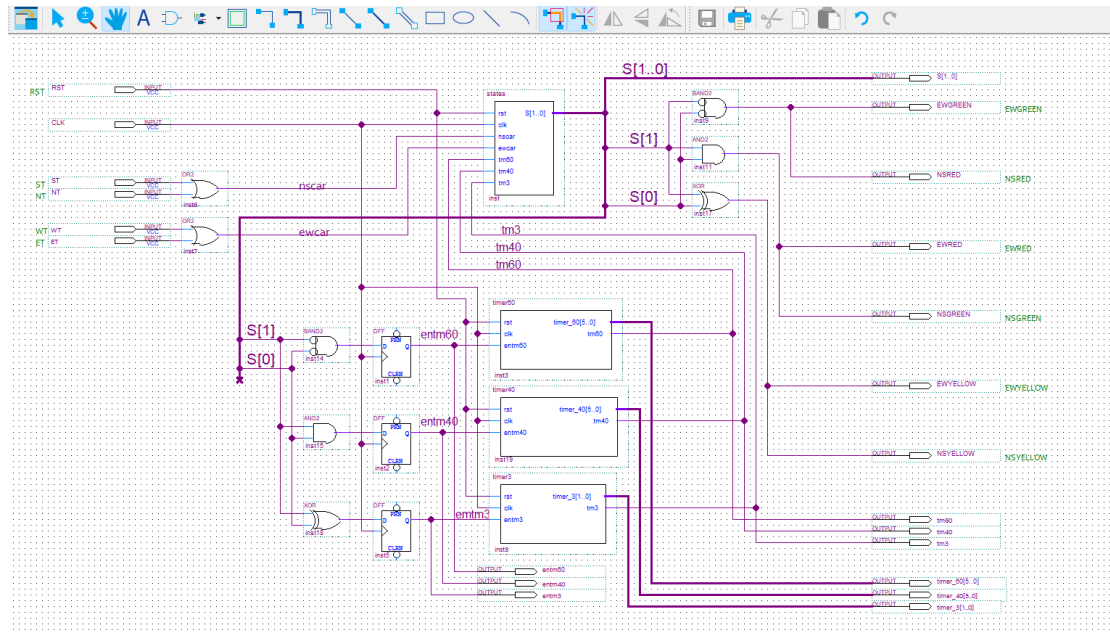
接着仿真双向车道都有车时的情况，仿真结果如下图：



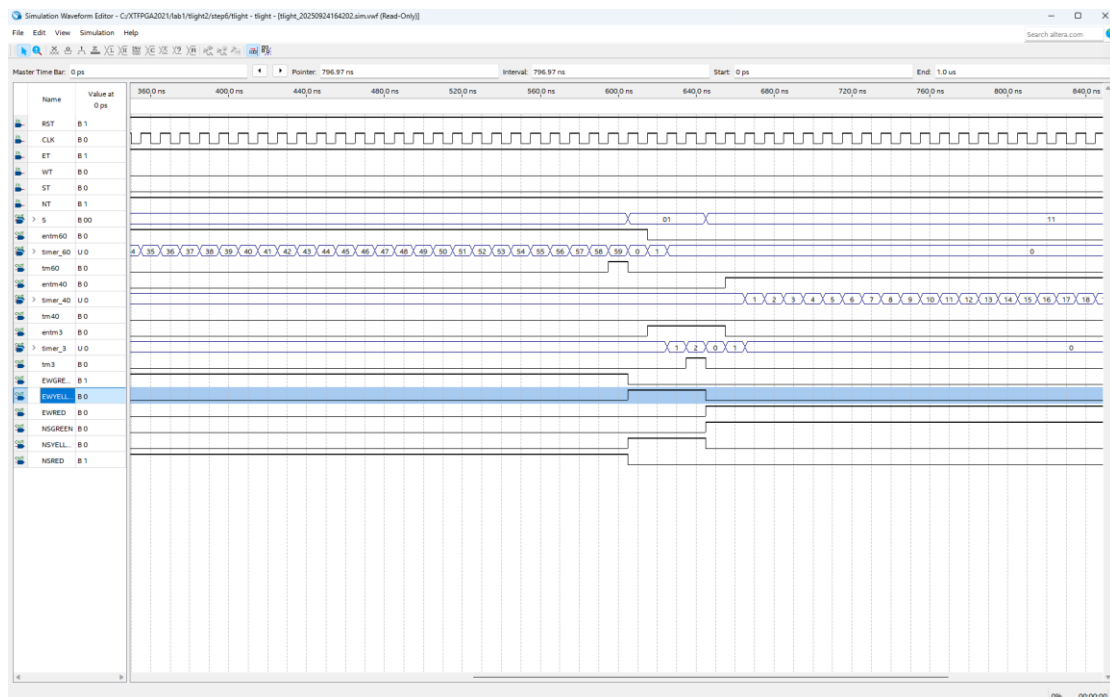
东西向和南北向均有车时，由东西向车道先行，南北向车道后行。在东西向车道由绿灯变黄灯（EWGREEN：1 到 0——EWYELLOW：0 到 1）时，发现在黄灯的变换上出了问题，本应该是三秒计数的黄灯，却在转换时持续了五拍的时间。

4. 在 3 的基础上改正设计。

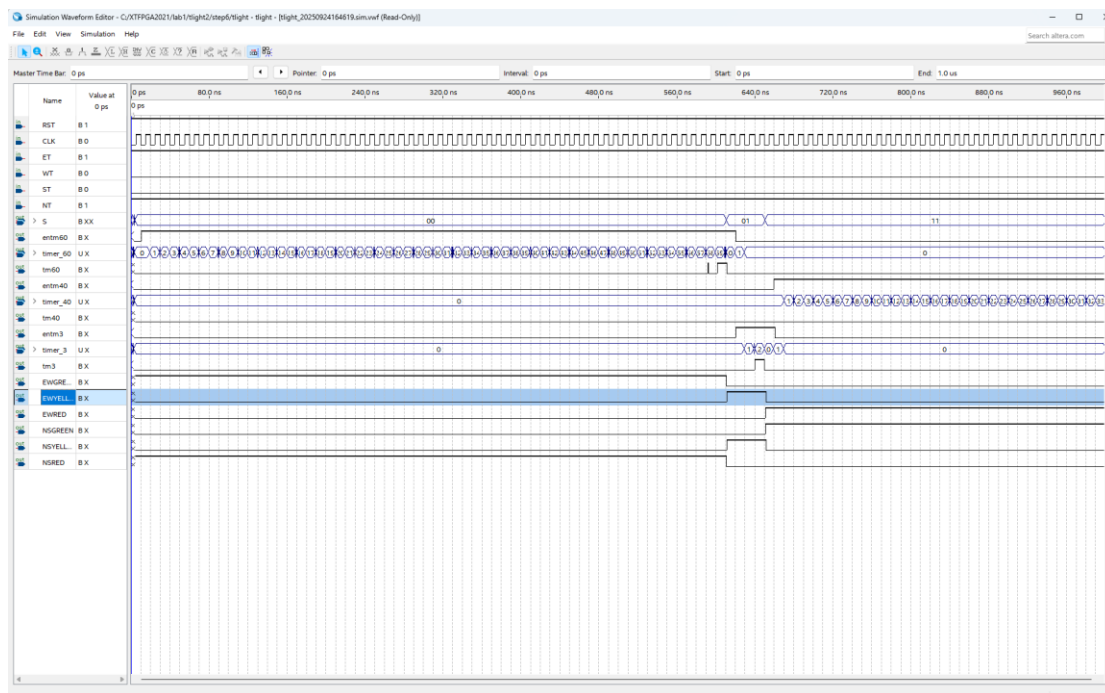
把 D 触发器删去，来减缓延迟效应。



功能仿真结果:



时序仿真结果:



把计数器前的三个用来获取 entm60、entm40、entm3 三个信号的 D 触发器也删除即可消除延迟。

5. 对该交通灯模型进行优化。

- 1) 通过改变模块输出的 D 触发器逻辑来减少延迟。
- 2) 通过把三个计数器合成为一个可变模计数器来简化模型。

三、实验结果与分析

1. 仿真时，应该按照什么顺序进行？

应该按照从简单情况到复杂情况的仿真顺序，逐渐地研究和对比仿真结果图，在覆盖所有种情况下全面地分析和检查设计是否有误。具体到交通灯仿真实验，就应该从只有单边有车，到双边都有车，到一边车道发生状态变换时同时改变另一车道状态。

2. 为什么状态转换的时间有问题？ 多出的两个时钟周期时怎么来的？

状态转换的时间有问题，是因为计数器的状态输出给了 D 触发器，再由 D 触

发器的输出使状态机转换 s 的状态。由于 D 触发器的状态改变依靠时钟信号的上升沿,造成了由 D 触发器的输出控制状态机就会出现多出来的两个时钟周期。

具体到仿真图形里,当 timer_60 信号计时到 59 时,下一个时钟上升沿 tm60 状态变为 1,同时 timer_60 信号计时变为 0,意味着绿灯 60 秒计时结束,即将转换为黄灯。下一个状态,tm60 变为 0,状态机转换为 s1。下一个状态改变时,entm60 变为 0。entm3 变为 1,黄灯亮起。经过三个时间周期后, timer_3 计时到 0,此时理应发生状态改变,但由于 tm3 和状态机 s 要到下一个时钟上升沿才能改变,下下一个时钟上升沿 entm40 才能变为 1,导致延迟了两个时钟周期。

3. 有几种解决方法?

- 1) 直接删除触发器连接。
- 2) 将三个计数器合并为一个可变模计数器,减少计数器之间的协调和延迟。
- 3) 额外设计新的同步信号,确保状态转换正确和计数器输出无延迟。

4. 不同的方法各自的特点是什么?

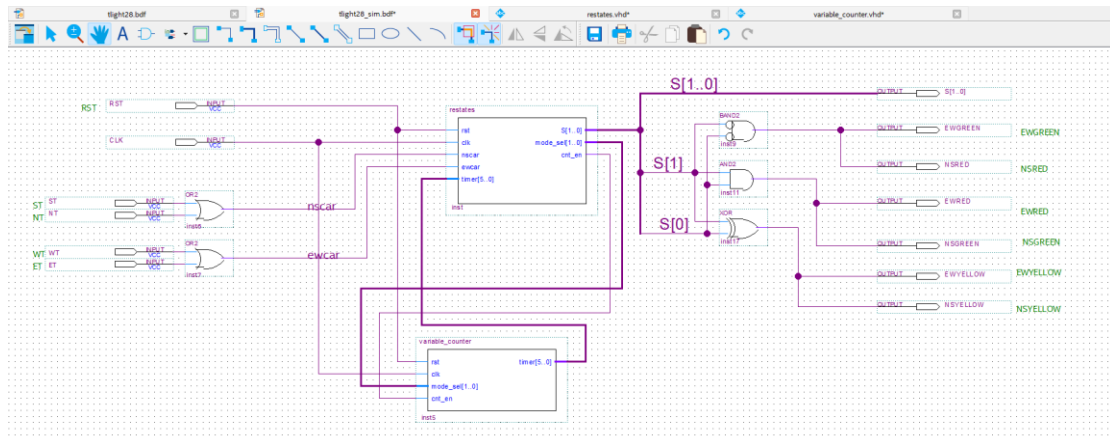
直接删除 D 触发器实现简单,直接消除延迟源,资源消耗少。设计新的时序逻辑来产生同步信号则实现困难,需要进行大规模的改动和调试。

5. 如何能简化这个设计?

将三个模计数器合成为一个可变模计数器

四、思考题

1. 画出你理想的红绿灯的电路系统框图,越细化越好。



2. 写出可变模计数器的完整 VHD 代码和 Verilog 代码。

在状态机中引入一个模式选择信号 mode_sel 和计数使能信号 cnt_en, 作为对可变模计数器的输入。

```

library IEEE;
use IEEE.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;

entity restates is
port (
    rst      : in std_logic;
    clk      : in std_logic;
    nscar    : in std_logic;
    ewcar    : in std_logic;
    timer    : in std_logic_vector(5 downto 0); -- 计时器输入(0-59)
    s        : out std_logic_vector(1 downto 0); -- 状态输出
    mode_sel : out std_logic_vector(1 downto 0); -- 计数器模式选择
    cnt_en   : out std_logic;                  -- 计数器使能信号
);
end restates;

architecture restates_behav of restates is
    constant s0 : std_logic_vector(1 downto 0) := "00";
    constant s1 : std_logic_vector(1 downto 0) := "01";
    constant s2 : std_logic_vector(1 downto 0) := "11";
    constant s3 : std_logic_vector(1 downto 0) := "10";
    -- 计数器模式
    constant MODE_3SEC  : std_logic_vector(1 downto 0) := "00"; -- 3秒模式
    constant MODE_40SEC : std_logic_vector(1 downto 0) := "01"; -- 40秒模式
    constant MODE_60SEC : std_logic_vector(1 downto 0) := "10"; -- 60秒模式
    signal ss : std_logic_vector(1 downto 0);
begin

```

可变模计数器代码：

```

library IEEE;

use IEEE.std_logic_1164.all;

use ieee.std_logic_arith.all;

use ieee.std_logic_unsigned.all;

entity variable_counter is

port (

```

```

rst      : in std_logic;

clk      : in std_logic;

mode_sel : in std_logic_vector(1 downto 0); -- 模式选择

cnt_en   : in std_logic;                    -- 计数使能

timer    : out std_logic_vector(5 downto 0) -- 计时器输出

);

end variable_counter;

architecture variable_counter_behav of variable_counter is

    signal count_reg : std_logic_vector(5 downto 0); -- 计数寄存器

    signal max_count : integer range 0 to 59;        -- 最大计数值

begin

    -- 根据模式选择设置最大计数值

    process(mode_sel)

    begin

        case mode_sel is

            when "00" => -- 3 秒模式

                max_count <= 2; -- 0-2 共 3 个周期

            when "01" => -- 40 秒模式

                max_count <= 39; -- 0-39 共 40 个周期

            when "10" => -- 60 秒模式

                max_count <= 59; -- 0-59 共 60 个周期

            when others => -- 默认 3 秒

```

```

        max_count <= 2;

    end case;

end process;

-- 计数进程

process(rst, clk)

begin

    if rst = '0' then

        count_reg <= (others => '0');

    elsif clk'event and clk = '1' then

        if cnt_en = '1' then

            if count_reg = max_count then

                count_reg <= (others => '0');  -- 计数满后归零

            else

                count_reg <= count_reg + 1;    -- 正常计数

            end if;

        else

            count_reg <= (others => '0');      -- 使能无效时清零

        end if;

    end if;

end process;

timer <= count_reg;  -- 输出当前计数

end variable_counter_behav;

```

可变模计数器 Verilog 代码:

```
module variable_timer(

    input rst,

    input clk,

    input [1:0] mode_sel,

    input cnt_en,

    output reg [5:0] timer

);

    reg [5:0] count_reg;

    integer max_count;

    // 根据模式选择设置最大计数值

    always @(*) begin

        case (mode_sel)

            2'b00:    max_count = 2;    // 3 秒模式: 0-2 共 3 个周期

            2'b01:    max_count = 39;   // 40 秒模式: 0-39 共 40 个周期

            2'b10:    max_count = 59;   // 60 秒模式: 0-59 共 60 个周期

            default: max_count = 2;     // 默认 3 秒模式

        endcase

    end

    // 计数进程

    always @(posedge clk or posedge rst) begin

        if (rst) begin
```

```

        // 复位时计数器清零

        count_reg <= 6'b000000;

    end else begin

        if (cnt_en) begin

            // 使能有效时计数

            if (count_reg == max_count) begin

                count_reg <= 6'b000000; // 计数满后归零

            end else begin

                count_reg <= count_reg + 1; // 正常计数

            end

        end else begin

            // 使能无效时清零

            count_reg <= 6'b000000;

        end

    end

end

always @(*) begin

    timer = count_reg;

end

endmodule

```

3. 参考网络资料，分析本例子中状态机 SS 的代码书写的好处和不好处，什么是状态机的 1、2、3 段式写法。将红绿灯的状态机 SS 用两段式、三段式代码用

VHDL 分别重写。

状态机的三段式写法是一种规范的状态机编写方法，将状态机分为三个部分：

第一段：状态寄存器（时序逻辑），负责状态寄存器的更新，通常在时钟沿或复位信号下更新当前状态。

第二段：下一状态逻辑（组合逻辑），根据当前状态和输入信号，计算下一状态。

第三段：输出逻辑（组合逻辑或时序逻辑），根据当前状态（有时也包含输入）产生输出信号。输出逻辑可以是组合逻辑也可以是时序逻辑。组合逻辑输出可能产生毛刺，时序逻辑输出则可以避免毛刺。

两段式写法，通常将第二段和第三段合并为一个组合逻辑进程，即下一状态逻辑和输出逻辑放在一个进程中。

两段式：

```
library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

use IEEE.STD_LOGIC_ARITH.ALL;

use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity states_2stage is

    port (

        rst      : in std_logic;

        clk      : in std_logic;

        nscar    : in std_logic;
```

```

        ewcar    : in std_logic;

        tm60     : in std_logic;

        tm40     : in std_logic;

        tm3      : in std_logic;

        S        : out std_logic_vector(1 downto 0)

    );

end states_2stage;

architecture Behavioral of states_2stage is

    -- 状态定义

    type state_type is (s0, s1, s2, s3);

    signal current_state, next_state : state_type;

begin

    -- 第一段：状态寄存器（时序逻辑）

    process(rst, clk)

    begin

        if rst = '0' then

            current_state <= s0;

        elsif rising_edge(clk) then

            current_state <= next_state;

        end if;

    end process;

    -- 第二段：下一状态逻辑和输出逻辑（组合逻辑）

```

```
process(current_state, nscar, ewcar, tm60, tm40, tm3)
```

```
begin
```

```
-- 默认下一状态和输出
```

```
next_state <= current_state;
```

```
S <= "00"; -- 默认输出，可根据状态赋值
```

```
case current_state is
```

```
when s0 =>
```

```
    S <= "00"; -- 代表 s0 状态输出
```

```
    if nscar = '1' then
```

```
        if ewcar = '1' then
```

```
            if tm60 = '1' then
```

```
                next_state <= s1;
```

```
            end if;
```

```
        else
```

```
            next_state <= s1;
```

```
        end if;
```

```
    end if;
```

```
when s1 =>
```

```
    S <= "01";
```

```
    if tm3 = '1' then
```

```
        next_state <= s2;
```

```
    end if;
```



```

when s2 =>

    S <= "11";

    if nscar = '1' then

        if ewcar = '1' then

            if tm40 = '1' then

                next_state <= s3;

            end if;

        else

            next_state <= s3;

        end if;

    end if;

when s3 =>

    S <= "10";

    if tm3 = '1' then

        next_state <= s0;

    end if;

when others =>

    next_state <= s0;

    S <= "00";

end case;

end process;

end Behavioral;

```

三段式写法：

```
library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

use IEEE.STD_LOGIC_ARITH.ALL;

use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity states_3stage is

    port (

        rst      : in std_logic;

        clk      : in std_logic;

        nscar    : in std_logic;

        ewcar    : in std_logic;

        tm60     : in std_logic;

        tm40     : in std_logic;

        tm3      : in std_logic;

        S        : out std_logic_vector(1 downto 0)

    );

end states_3stage;

architecture Behavioral of states_3stage is

    type state_type is (s0, s1, s2, s3);

    signal current_state, next_state : state_type;

begin

    -- 第一段：状态寄存器（时序逻辑）
```

```

process(rst, clk)

begin

    if rst = '0' then

        current_state <= s0;

    elsif rising_edge(clk) then

        current_state <= next_state;

    end if;

end process;

-- 第二段： 下一状态逻辑（组合逻辑）

process(current_state, nscar, ewcar, tm60, tm40, tm3)

begin

    next_state <= current_state; -- 默认保持当前状态

    case current_state is

        when s0 =>

            if nscar = '1' then

                if ewcar = '1' then

                    if tm60 = '1' then

                        next_state <= s1;

                    end if;

                else

                    next_state <= s1;

                end if;

            end if;

```

```
        end if;

    when s1 =>

        if tm3 = '1' then

            next_state <= s2;

        end if;

    when s2 =>

        if nscar = '1' then

            if ewcar = '1' then

                if tm40 = '1' then

                    next_state <= s3;

                end if;

            else

                next_state <= s3;

            end if;

        end if;

    when s3 =>

        if tm3 = '1' then

            next_state <= s0;

        end if;

    when others =>

        next_state <= s0;

    end case;
```

```
end process;

-- 第三段：输出逻辑（组合逻辑）

process(current_state)

begin

    case current_state is

        when s0 => S <= "00";

        when s1 => S <= "01";

        when s2 => S <= "11";

        when s3 => S <= "10";

        when others => S <= "00";

    end case;

end process;

end Behavioral;
```